

Team Process & Management Document

CSE 3902 Hollow Knight
Team Pie

Prepared by: Nicholas Mamais(Project Manager)
The Ohio State University Spring 2026

1. Overview

This document describes the development process, team management practices, and quality assurance workflow for our Hollow Knight clone project built with MonoGame and C#. It covers our branching strategy, pull request review process, code feedback standards, sprint workflow, and communication practices.

2. Team Structure & Roles

Our team operates with shared ownership of the codebase. While every member contributes across the project, individuals take primary responsibility for specific subsystems to ensure accountability and deep expertise.

Role	Responsibility	Description
Project Lead	Coordination	Manages sprint planning, resolves blockers, and ensures deliverables are on track.
Developer	Implementation	Implements features in assigned subsystems (sprites, controllers, game logic, etc.).
Reviewer	Quality Assurance	All team members rotate as PR reviewers. Every PR requires two approvals before merge.

3. Branching Strategy

We follow a feature-branch workflow with a protected main branch. This keeps the main branch stable and deployable at all times while giving developers isolated environments to build and test features.

3.1 Branch Naming Convention

All branches follow a consistent naming pattern to make their purpose immediately clear:

Prefix	Purpose	Example
feature/	New functionality	feature/player-movement
bugfix/	Bug repairs	bugfix/sprite-flicker
refactor/	Code restructuring	refactor/command-pattern
sprint/	Sprint integration	sprint/sprint1-integration

3.2 Branch Lifecycle

1. Create a new branch from main (or the current sprint integration branch) for each task or feature.
2. Develop and commit locally with clear, descriptive commit messages.
3. Push the branch to the remote repository and open a pull request.
4. After two approved reviews, the branch is merged and deleted.

3.3 Protected Branches

The main branch is protected with the following rules:

- Direct pushes to main are not allowed, same goes for dev.
- All changes must come through a pull request.
- A minimum of two approving reviews are required before merging.
- The branch must be up to date with main before merging (no stale merges).

4. Pull Request (PR) Review Process

Pull requests are the primary quality gate in our workflow. Every change to the codebase—no matter how small, goes through the PR process.

4.1 PR Submission Requirements

When opening a PR, the author must include:

- A clear title summarizing the change (e.g., “Add keyboard controller with command pattern”).
- A description explaining what was changed and why.
- References to any related issues or sprint tasks.
- A self-review checklist confirming the code compiles, runs, and has been tested locally.

4.2 Two-Reviewer Requirement

Every PR must be approved by at least two team members before it can be merged. Reviewers are assigned on a rotating basis, though any team member may volunteer to review. This ensures:

- Multiple perspectives catch different types of issues.
- Knowledge of the codebase is distributed across the team.
- No single person becomes a bottleneck for merges.

4.3 Review Turnaround

Reviewers are expected to complete their review within 24 hours of assignment. If a reviewer cannot meet this window, they should notify the team so another reviewer can step in. Keeping reviews timely prevents work from stalling and avoids large merge conflicts.

4.4 Merge Policy

Once two approvals are received and any requested changes have been addressed, the PR author merges their own branch. We use squash merges to keep the main branch history clean and readable.

5. Code Review & Feedback Standards

We treat code review as a collaborative learning process, not just a quality check. Feedback should be constructive, specific, and actionable.

5.1 What Reviewers Look For

Category	Details
Correctness	Does the code do what it claims? Are there edge cases or off-by-one errors?
Design Patterns	Does it follow our established patterns (Command, Factory, Singleton, ISprite interface)?
Readability	Are names descriptive? Is the code easy to follow without excessive comments?
Architecture	Does it fit into the existing project structure (Controllers, Commands, Sprites, etc.)?
Performance	Are there unnecessary allocations in the game loop? Is Update/Draw efficient?
XML Docs	Do public classes and methods have summary comments?

5.2 Feedback Format

We categorize review comments to set clear expectations for the author:

Must Fix: Issues that block the merge, bugs, broken patterns, missing null checks, compilation errors.

Should Fix: Improvements strongly recommended before merge, naming, documentation gaps, minor design concerns.

Suggestion: Optional improvements that can be addressed in a follow-up—refactoring ideas, performance optimizations, style preferences.

Praise: Highlight good work. Positive feedback reinforces good practices and keeps morale up.

5.3 Tone & Communication

All review feedback should be respectful and focused on the code, not the person. We follow these principles:

- Ask questions rather than make demands (“Would it make sense to extract this into a method?” vs. “Extract this.”).
- Explain the “why” behind feedback so the author learns, not just fixes.
- Offer alternative solutions when pointing out problems.

- Acknowledge when you're unsure, it's okay to say "I think this might cause an issue, but I'm not certain."

6. Sprint Workflow

We work in weekly sprints aligned with the course deliverables. Each sprint follows a predictable rhythm:

6.1 Sprint Cadence

Day	Activity
Monday	Sprint planning: assign tasks, create branches, set goals for the week.
Tue–Thu	Development: implement features, open PRs, conduct reviews.
Friday	Integration & testing: merge remaining PRs, integration testing, bug fixes.
Weekend	Buffer: address any remaining issues, prepare for sprint demo if applicable.

6.2 Task Tracking

Tasks are tracked using GitHub Issues. Each issue includes a clear description, acceptance criteria, and an assignee. Issues are linked to PRs so progress is visible at a glance.

7. Communication

Effective communication is essential for a team project with parallel development. Our communication channels serve different purposes:

GitHub PR Comments: Primary channel for all code-related discussion. Keeps feedback tied directly to the code in question.

Team Group Chat: Quick questions, status updates, blockers, and coordination. Used for day-to-day communication.

Weekly Meetings: Sprint planning, retrospectives, and design discussions. Decisions made here are documented in meeting notes.

8. Coding Standards

To maintain consistency across the codebase, we follow these conventions:

- C# naming conventions: PascalCase for public members, `_camelCase` with underscore prefix for private fields.
- All public classes and methods include XML documentation comments (summary tags).
- Interfaces are prefixed with "I" (ISprite, ICommand, IController).
- One class per file, file name matches class name.

- Namespaces mirror the folder structure (HollowKnight.Sprites, HollowKnight.Commands, etc.).
- No magic numbers—use named constants or parameters with default values.
- Game loop methods (Update/Draw) should be kept lightweight; delegate complex logic to subsystems.

9. Merge Conflict Resolution

When merge conflicts arise, we follow a clear process to avoid losing work:

1. The developer whose branch has the conflict is responsible for resolving it.
2. Pull the latest main into the feature branch locally.
3. Resolve conflicts carefully, preserving both sets of changes where possible.
4. Test the resolved code locally before pushing.
5. If the conflict involves another developer's work, coordinate with them directly before resolving.

10. Continuous Improvement

At the end of each sprint, the team conducts a brief retrospective to reflect on what went well, what didn't, and what to improve. Common topics include PR turnaround times, code quality trends, and communication effectiveness. Action items from retrospectives are tracked and reviewed the following week.

This living document is updated as our process evolves. All team members are encouraged to propose changes to improve our workflow.